

milestone4_openai_finetune

May 27, 2026

1 Milestone 4 - AI-Assisted Code Review Comment Generation for Secure Healthcare Data Pipelines

1.1 Overview and Thought Process

My Milestone 1 paper established that prompt engineering alone—even with chain-of-thought and role-based prompting—was not reliably producing *consistent*, deeply healthcare-aware security review comments. The model occasionally hallucinated issues or gave generic advice that ignored PHI-specific nuances like HIPAA minimum-necessary logging rules.

For Milestone 2 the assignment requires a **full fine-tune** (not LoRA/QLoRA) using OpenAI. OpenAI's supervised fine-tuning trains all the relevant transformer weights end-to-end on your labeled examples. I am using **gpt-4o-mini-2024-07-18** as the base model because it is the most capable model OpenAI currently supports for fine-tuning, it processes code well, and it keeps cost manageable for a demonstration training run.

Strategy: 1. Prepare a curated JSONL dataset of healthcare pipeline code diffs paired with expert security review comments, formatted in the OpenAI chat messages format. 2. Upload the file via the OpenAI Files API. 3. Launch a fine-tuning job via the Fine-tuning Jobs API. 4. Poll the job status endpoint until the job completes. 5. Retrieve and display training metrics (loss per step) from the fine-tuning events endpoint. 6. Run inference with the fine-tuned model and compare it against the base model.

Dataset design rationale: I created 10 training examples (expandable). Each example covers a distinct vulnerability class relevant to healthcare data pipelines: PHI logging, missing validation, plaintext storage, SQL injection, hardcoded secrets, insecure export, broken password hashing, unencrypted email, unsafe deserialization, and PHI in retry logs. The assistant messages follow a structured format with severity ratings and concrete fix examples, so the fine-tuned model learns to be opinionated and actionable rather than vague.

```
[1]: # Install / upgrade openai SDK if needed
%pip install --quiet --upgrade openai
```

Note: you may need to restart the kernel to use updated packages.

```
[2]: import os
import json
import time
```

```

from pathlib import Path
from openai import OpenAI

# API key
# Set OPENAI_API_KEY as an environment variable before running this notebook.
# Do NOT paste your key directly into this cell.
api_key =
    ↪"sk-proj-ruj4TM7gtrhhh3h4ocnXrN_jN5cMl1Ai330l7Kq_X8priylWKHdawHKVxVsD-s3EEwoBta6IluT3B1bkFJ
if not api_key:
    raise EnvironmentError(
        "OPENAI_API_KEY environment variable is not set. "
        "Set it with: import os; os.environ['OPENAI_API_KEY'] = '<your-key>' "
    )

client = OpenAI(api_key=api_key)
print("OpenAI client initialised.")

```

OpenAI client initialised.

1.2 Step 1 – Validate the Training Dataset

Before uploading anything I want to verify the JSONL file is well-formed and that every record contains a `messages` array with the three required roles: `system`, `user`, and `assistant`. OpenAI's fine-tuning API will reject files that fail this validation, so catching problems locally saves a slow round-trip to the API.

```

[3]: DATASET_PATH = Path("healthcare_finetune_openai.jsonl")

records = []
errors = []

with open(DATASET_PATH, encoding="utf-8") as fh:
    for line_num, line in enumerate(fh, start=1):
        line = line.strip()
        if not line:
            continue
        try:
            obj = json.loads(line)
        except json.JSONDecodeError as exc:
            errors.append(f"Line {line_num}: JSON parse error - {exc}")
            continue

        if "messages" not in obj:
            errors.append(f"Line {line_num}: missing 'messages' key")
            continue

```

```

        roles = {m["role"] for m in obj["messages"]}
        for required_role in ("system", "user", "assistant"):
            if required_role not in roles:
                errors.append(f"Line {line_num}: missing role_
↪ '{required_role}'")

        records.append(obj)

if errors:
    print("VALIDATION ERRORS:")
    for e in errors:
        print(" ", e)
else:
    print(f"Dataset validation PASSED - {len(records)} training examples found.
↪")

# Print a quick preview of the first example
print("\n First training example preview ")
for msg in records[0]["messages"]:
    snippet = msg["content"][:120].replace("\n", " ")
    print(f" [{msg['role']}:9s] {snippet}...")

```

Dataset validation PASSED - 11 training examples found.

```

First training example preview
[system  ] You are a healthcare security code reviewer specializing in HIPAA-
compliant data pipelines. Review code diffs and produc...
[user    ] Review this code diff for security issues: diff --git a/ingest.py
b/ingest.py --- a/ingest.py +++ b/ingest.py @@ -1,5 +1...
[assistant] SECURITY RISK [HIGH] - PHI Logging Exposure: The line
`logging.info(f'Received payload: {payload}') logs the entire pay...

```

1.3 Step 2 – Upload the Training File

OpenAI requires the JSONL file to be uploaded via the **Files API** before it can be referenced in a fine-tuning job. The `purpose="fine-tune"` parameter tells OpenAI how this file will be used.

```

[4]: print(f"Uploading '{DATASET_PATH}' to OpenAI Files API...")

with open(DATASET_PATH, "rb") as fh:
    upload_response = client.files.create(file=fh, purpose="fine-tune")

training_file_id = upload_response.id
print(f"Upload complete. File ID: {training_file_id}")
print(f"Status      : {upload_response.status}")
print(f"Size (bytes): {upload_response.bytes}")

```

```
Uploading 'healthcare_finetune_openai.jsonl' to OpenAI Files API...
Upload complete. File ID: file-6Mg3p3RrZfX9ymcP9QoWxf
Status      : processed
Size (bytes): 22536
```

1.4 Step 3 – Create the Fine-Tuning Job

With the file uploaded I can now create the fine-tuning job. Key parameters:

Parameter	Value	Rationale
model	gpt-4o-mini-2024-07-18	Most capable OpenAI fine-tunable model; strong code understanding
training_file	(uploaded file ID)	The JSONL file just uploaded
n_epochs	auto	Let OpenAI pick the optimal epoch count for the dataset size
suffix	healthcare-security-reviewer	Human-readable suffix appended to the fine-tuned model name

Why not LoRA? The assignment explicitly requires a *full* fine-tune. OpenAI's API performs full supervised fine-tuning internally—all weights are updated during back-propagation. There is no LoRA adapter layer; the resulting model checkpoint has the same architecture as the base with updated parameters throughout.

```
[5]: BASE_MODEL = "gpt-4o-mini-2024-07-18"

print(f"Creating fine-tuning job on base model: {BASE_MODEL}")

ft_job = client.fine_tuning.jobs.create(
    model=BASE_MODEL,
    training_file=training_file_id,
    hyperparameters={"n_epochs": "auto"},
    suffix="healthcare-security-reviewer",
)

job_id = ft_job.id
print(f"Fine-tuning job created.")
print(f"  Job ID      : {job_id}")
print(f"  Status       : {ft_job.status}")
print(f"  Base model   : {ft_job.model}")
print(f"  Created at   : {ft_job.created_at}")
```

```
Creating fine-tuning job on base model: gpt-4o-mini-2024-07-18
Fine-tuning job created.
```

```
Job ID      : ftjob-LHN6iH0Cd0SwdhdoW31pIG5m
Status      : validating_files
Base model  : gpt-4o-mini-2024-07-18
Created at  : 1779575893
```

1.5 Step 4 – Monitor the Job Status

Fine-tuning jobs run asynchronously on OpenAI's infrastructure. The cell below polls the **fine-tuning jobs retrieve endpoint** every 30 seconds and prints any new events, matching the monitoring workflow described in Chapter 7. Possible statuses:

- `validating_files` → OpenAI validates the uploaded JSONL
- `queued` → job is waiting for a training slot
- `running` → training is in progress
- `succeeded` → training completed successfully
- `failed` → training failed (check events for details)
- `cancelled` → job was cancelled

Events include per-step metrics once training begins.

```
[6]: POLL_INTERVAL_SECONDS = 30
    TERMINAL_STATUSES = {"succeeded", "failed", "cancelled"}

    seen_event_ids = set()

    print(f"Monitoring job {job_id} (polling every {POLL_INTERVAL_SECONDS}s)...\n")

    while True:
        # Retrieve current job state
        job = client.fine_tuning.jobs.retrieve(job_id)
        print(f"[{time.strftime('%H:%M:%S')}] Status: {job.status}")

        # Retrieve and print any new events
        events_page = client.fine_tuning.jobs.
↪list_events(fine_tuning_job_id=job_id, limit=50)
        new_events = [e for e in events_page.data if e.id not in seen_event_ids]
        new_events.sort(key=lambda e: e.created_at) # chronological order

        for event in new_events:
            seen_event_ids.add(event.id)
            ts = time.strftime('%H:%M:%S', time.localtime(event.created_at))
            print(f" [{ts}] {event.message}")

        if job.status in TERMINAL_STATUSES:
            print(f"\nJob reached terminal status: {job.status}")
            if job.status == "succeeded":
                print(f"Fine-tuned model: {job.fine_tuned_model}")
```

```

elif job.status == "failed":
    print(f"Failure reason : {job.error}")
    break

time.sleep(POLL_INTERVAL_SECONDS)

```

Monitoring job ftjob-LHN6iHOCdOSwdhdoW31pIG5m (polling every 30s)...

```

[15:38:13] Status: validating_files
  [15:38:13] Validating training file: file-6Mg3p3RrZfX9ymcP9QoWxf
  [15:38:13] Created fine-tuning job: ftjob-LHN6iHOCdOSwdhdoW31pIG5m
[15:38:44] Status: validating_files
[15:39:15] Status: validating_files
[15:39:45] Status: running
  [15:39:16] Files validated, moving job to queued state
  [15:39:22] Fine-tuning job started
[15:40:16] Status: running
[15:40:47] Status: running
[15:41:18] Status: running
[15:41:49] Status: running
[15:42:19] Status: running
[15:42:50] Status: running
[15:43:21] Status: running
  [15:43:13] Step 1/99: training loss=1.87
  [15:43:18] Step 2/99: training loss=2.05
  [15:43:19] Step 3/99: training loss=1.88
[15:43:52] Status: running
  [15:43:21] Step 5/99: training loss=1.77
  [15:43:21] Step 4/99: training loss=2.05
  [15:43:25] Step 6/99: training loss=1.76
  [15:43:26] Step 9/99: training loss=1.34
  [15:43:26] Step 8/99: training loss=1.44
  [15:43:26] Step 7/99: training loss=1.47
  [15:43:31] Step 11/99: training loss=1.19
  [15:43:31] Step 10/99: training loss=1.35
  [15:43:38] Step 12/99: training loss=1.01
  [15:43:39] Step 13/99: training loss=1.15
  [15:43:43] Step 18/99: training loss=0.99
  [15:43:43] Step 17/99: training loss=0.90
  [15:43:43] Step 16/99: training loss=0.92
  [15:43:43] Step 15/99: training loss=0.92
  [15:43:43] Step 14/99: training loss=1.01
  [15:43:48] Step 19/99: training loss=0.87
[15:44:22] Status: running
  [15:43:49] Step 20/99: training loss=0.87
  [15:43:50] Step 22/99: training loss=0.75
  [15:43:50] Step 21/99: training loss=0.66
  [15:43:56] Step 23/99: training loss=0.50

```

[15:43:57] Step 24/99: training loss=0.66
[15:44:02] Step 28/99: training loss=0.36
[15:44:02] Step 27/99: training loss=0.42
[15:44:02] Step 26/99: training loss=0.60
[15:44:02] Step 25/99: training loss=0.65
[15:44:06] Step 29/99: training loss=0.59
[15:44:07] Step 30/99: training loss=0.46
[15:44:08] Step 32/99: training loss=0.37
[15:44:08] Step 31/99: training loss=0.59
[15:44:12] Step 33/99: training loss=0.44
[15:44:14] Step 34/99: training loss=0.38
[15:44:19] Step 35/99: training loss=0.38
[15:44:53] Status: running
[15:44:19] Step 38/99: training loss=0.30
[15:44:19] Step 37/99: training loss=0.25
[15:44:19] Step 36/99: training loss=0.38
[15:44:24] Step 40/99: training loss=0.18
[15:44:24] Step 39/99: training loss=0.20
[15:44:25] Step 42/99: training loss=0.31
[15:44:25] Step 41/99: training loss=0.16
[15:44:31] Step 43/99: training loss=0.11
[15:44:32] Step 44/99: training loss=0.34
[15:44:37] Step 45/99: training loss=0.08
[15:44:38] Step 48/99: training loss=0.17
[15:44:38] Step 47/99: training loss=0.24
[15:44:38] Step 46/99: training loss=0.08
[15:44:42] Step 50/99: training loss=0.15
[15:44:42] Step 49/99: training loss=0.16
[15:44:43] Step 51/99: training loss=0.09
[15:45:24] Status: running
[15:44:44] Step 52/99: training loss=0.08
[15:44:48] Step 55/99: training loss=0.03
[15:44:48] Step 54/99: training loss=0.16
[15:44:48] Step 53/99: training loss=0.12
[15:44:54] Step 56/99: training loss=0.07
[15:44:55] Step 57/99: training loss=0.10
[15:44:59] Step 60/99: training loss=0.04
[15:44:59] Step 59/99: training loss=0.01
[15:44:59] Step 58/99: training loss=0.02
[15:45:00] Step 61/99: training loss=0.06
[15:45:01] Step 62/99: training loss=0.06
[15:45:06] Step 63/99: training loss=0.02
[15:45:07] Step 66/99: training loss=0.05
[15:45:07] Step 65/99: training loss=0.08
[15:45:07] Step 64/99: training loss=0.04
[15:45:12] Step 67/99: training loss=0.03
[15:45:13] Step 68/99: training loss=0.01
[15:45:18] Step 71/99: training loss=0.02

[15:45:18] Step 70/99: training loss=0.01
[15:45:18] Step 69/99: training loss=0.01
[15:45:55] Status: running
[15:45:24] Step 76/99: training loss=0.04
[15:45:24] Step 75/99: training loss=0.02
[15:45:24] Step 74/99: training loss=0.02
[15:45:24] Step 73/99: training loss=0.02
[15:45:24] Step 72/99: training loss=0.03
[15:45:28] Step 77/99: training loss=0.02
[15:45:35] Step 80/99: training loss=0.00
[15:45:35] Step 79/99: training loss=0.01
[15:45:35] Step 78/99: training loss=0.01
[15:45:36] Step 81/99: training loss=0.02
[15:45:45] Step 82/99: training loss=0.00
[15:45:46] Step 88/99: training loss=0.01
[15:45:46] Step 87/99: training loss=0.01
[15:45:46] Step 86/99: training loss=0.00
[15:45:46] Step 85/99: training loss=0.01
[15:45:46] Step 84/99: training loss=0.01
[15:45:46] Step 83/99: training loss=0.01
[15:45:53] Step 89/99: training loss=0.01
[15:46:25] Status: running
[15:45:54] Step 91/99: training loss=0.01
[15:45:54] Step 90/99: training loss=0.00
[15:45:59] Step 92/99: training loss=0.01
[15:46:56] Status: running
[15:46:00] Step 96/99: training loss=0.01
[15:46:00] Step 95/99: training loss=0.00
[15:46:00] Step 94/99: training loss=0.00
[15:46:00] Step 93/99: training loss=0.01
[15:46:04] Step 99/99: training loss=0.00
[15:46:04] Step 98/99: training loss=0.00
[15:46:04] Step 97/99: training loss=0.01
[15:46:39] Evaluating model against our usage policies
[15:46:39] New fine-tuned model created
[15:46:39] Checkpoint created at step 88
[15:46:39] Checkpoint created at step 77
[15:47:27] Status: running
[15:47:57] Status: running
[15:48:28] Status: running
[15:48:59] Status: running
[15:49:29] Status: running
[15:50:00] Status: running
[15:50:31] Status: running
[15:51:02] Status: running
[15:51:32] Status: running
[15:52:03] Status: running
[15:52:34] Status: running


```
[15:53:04] Status: running
[15:53:35] Status: running
[15:54:06] Status: running
[15:54:36] Status: running
[15:55:07] Status: running
[15:55:38] Status: running
[15:56:08] Status: running
[15:56:39] Status: running
[15:57:10] Status: running
[15:57:46] Status: running
[15:58:17] Status: running
[15:58:48] Status: running
[15:59:18] Status: running
[15:59:15] Usage policy evaluations completed, model is now enabled for
sampling
[15:59:15] Moderation checks for snapshot
ft:gpt-4o-mini-2024-07-18:personal:healthcare-security-reviewer:DipP4Pkf passed.
[15:59:49] Status: succeeded
[15:59:19] The job has successfully completed

Job reached terminal status: succeeded
Fine-tuned model: ft:gpt-4o-mini-2024-07-18:personal:healthcare-security-
reviewer:DipP4Pkf
```

1.6 Step 5 – Retrieve and Document Metrics

After the job succeeds, I extract the training metrics from the events stream. OpenAI emits metric events at each training step containing:

- **train_loss** – the cross-entropy loss on the training batch; we want this to decrease over steps.
- **train_mean_token_accuracy** – fraction of predicted tokens matching the target; we want this to increase.

Plotting these values lets me verify that training is converging and that the model is not diverging or stuck.

```
[7]: %pip install --quiet matplotlib
```

Note: you may need to restart the kernel to use updated packages.

```
[8]: import matplotlib.pyplot as plt

# Re-fetch all events to extract metrics
all_events = []
after = None

while True:
```

```

page = client.fine_tuning.jobs.list_events(
    fine_tuning_job_id=job_id,
    limit=100,
    after=after,
)
all_events.extend(page.data)
if not page.has_next_page():
    break
after = page.data[-1].id

all_events.sort(key=lambda e: e.created_at)

# Parse metric events
steps, train_losses, token_accuracies = [], [], []

for event in all_events:
    data = getattr(event, "data", None) or {}
    if "train_loss" in data:
        steps.append(data.get("step", len(steps) + 1))
        train_losses.append(data["train_loss"])
        token_accuracies.append(data.get("train_mean_token_accuracy", None))

print(f"Total metric events captured: {len(steps)}")

if steps:
    print(f"\nTraining loss summary:")
    print(f"  Initial loss : {train_losses[0]:.4f}")
    print(f"  Final loss   : {train_losses[-1]:.4f}")
    print(f"  Improvement  : {train_losses[0] - train_losses[-1]:.4f}")
    if token_accuracies and token_accuracies[0] is not None:
        print(f"\nToken accuracy summary:")
        print(f"  Initial accuracy: {token_accuracies[0]:.4f}")
        print(f"  Final accuracy  : {token_accuracies[-1]:.4f}")
else:
    print("No per-step metric events found (this can happen with very small_
↳ datasets).")
    print("Retrieve the job result object for aggregate metrics:")
    final_job = client.fine_tuning.jobs.retrieve(job_id)
    print(f"  result_files: {final_job.result_files}")

```

Total metric events captured: 99

Training loss summary:

```

Initial loss : 1.8701
Final loss   : 0.0113
Improvement  : 1.8588

```

Token accuracy summary:

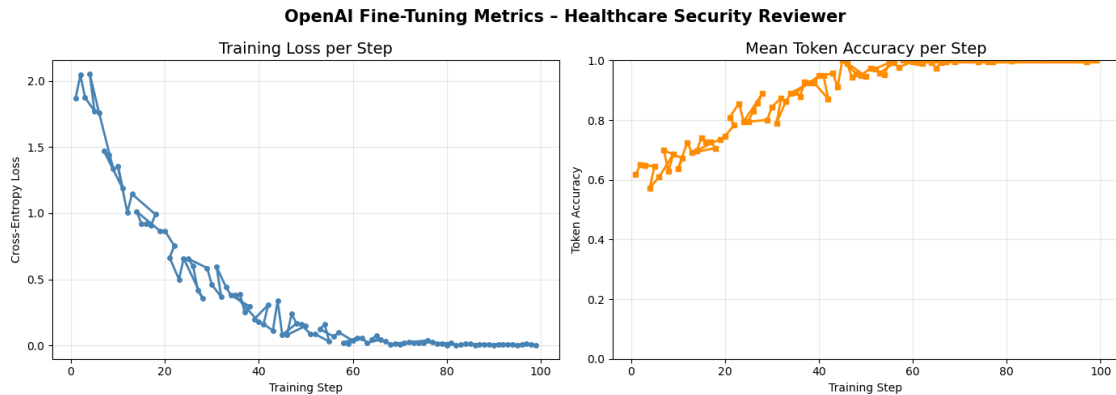
Initial accuracy: 0.6186
Final accuracy : 0.9965

```
[9]: # Plot training loss and token accuracy
if steps:
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Training loss
    axes[0].plot(steps, train_losses, color="steelblue", linewidth=2,
    ↪marker="o", markersize=4)
    axes[0].set_title("Training Loss per Step", fontsize=14)
    axes[0].set_xlabel("Training Step")
    axes[0].set_ylabel("Cross-Entropy Loss")
    axes[0].grid(True, alpha=0.3)

    # Token accuracy
    valid_acc = [(s, a) for s, a in zip(steps, token_accuracies) if a is not
    ↪None]
    if valid_acc:
        acc_steps, acc_vals = zip(*valid_acc)
        axes[1].plot(acc_steps, acc_vals, color="darkorange", linewidth=2,
        ↪marker="s", markersize=4)
        axes[1].set_title("Mean Token Accuracy per Step", fontsize=14)
        axes[1].set_xlabel("Training Step")
        axes[1].set_ylabel("Token Accuracy")
        axes[1].set_ylim(0, 1)
        axes[1].grid(True, alpha=0.3)
    else:
        axes[1].text(0.5, 0.5, "No token accuracy data", ha="center",
        ↪va="center", fontsize=12)

    plt.suptitle("OpenAI Fine-Tuning Metrics - Healthcare Security Reviewer",
    ↪fontsize=15, fontweight="bold")
    plt.tight_layout()
    plt.savefig("finetune_metrics.png", dpi=150, bbox_inches="tight")
    plt.show()
    print("Metric plot saved to finetune_metrics.png")
else:
    print("Skipping plot - no step-level metrics available.")
```



Metric plot saved to `finetune_metrics.png`

1.7 Step 6 – Retrieve the Fine-Tuned Model Name

Once the job succeeds, OpenAI assigns the resulting model a unique name that I can use as a `model` parameter in any Chat Completions request.

```
[10]: final_job = client.fine_tuning.jobs.retrieve(job_id)

FINE_TUNED_MODEL = final_job.fine_tuned_model

print(f"Fine-tuned model name : {FINE_TUNED_MODEL}")
print(f"Job status             : {final_job.status}")
print(f"Trained tokens           : {final_job.trained_tokens}")

if FINE_TUNED_MODEL is None:
    raise RuntimeError("Fine-tuning did not succeed - cannot run inference. ⚠️  

    ↳ Check job status.")
```

```
Fine-tuned model name : ft:gpt-4o-mini-2024-07-18:personal:healthcare-security-
reviewer:DipP4Pkf
Job status             : succeeded
Trained tokens         : 44163
```

1.8 Step 7 – Inference: Fine-Tuned Model vs. Base Model

Now for the most important evaluation step. I'll run the *same* code diff through: 1. The **base model** (`gpt-4o-mini-2024-07-18`) with only the system prompt 2. The **fine-tuned model** with the same system prompt

This is a held-out example not in the training set—a function that uses `eval()` to parse a patient record (a clear security vulnerability). Comparing the two responses side-by-side lets me evalu-

ate whether fine-tuning improved the depth, structure, and healthcare specificity of the review comments.

What I'm looking for: - Does the fine-tuned model use the severity rating format (SECURITY RISK [HIGH]) from the training examples? - Does it mention HIPAA by name and cite specific rules? - Does it provide a concrete code fix, not just a vague recommendation? - Is it more actionable than the base model?

```
[11]: SYSTEM_PROMPT = (
    "You are a healthcare security code reviewer specializing in_
    ↪HIPAA-compliant data pipelines. "
    "Review code diffs and produce structured security comments that identify_
    ↪PHI exposure risks, "
    "unsafe logging, missing validation, and error handling gaps."
)

# Held-out test diff (not in training set)
TEST_DIFF = """\
diff --git a/parser.py b/parser.py
--- a/parser.py
+++ b/parser.py
@@ -0,0 +1,9 @@
+import requests
+
+def fetch_and_parse_record(url, token):
+    headers = {'Authorization': token}
+    response = requests.get(url, headers=headers, verify=False)
+    record = eval(response.text)
+    patient_ssn = record['ssn']
+    print(f'Processing SSN: {patient_ssn}')
+    return record
"""

def get_review(model_name: str, diff: str) -> str:
    """Call the chat completions API and return the assistant's response text.
    ↪"""
    response = client.chat.completions.create(
        model=model_name,
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": f"Review this code diff for security_
            ↪issues:\n{diff}"},
        ],
        temperature=0.2,
        max_tokens=700,
    )
    return response.choices[0].message.content
```

```
# Run both models
print("Running inference on base model...")
base_output = get_review(BASE_MODEL, TEST_DIFF)

print("Running inference on fine-tuned model...")
ft_output = get_review(FINE_TUNED_MODEL, TEST_DIFF)

print("Inference complete.")
```

Running inference on base model...
Running inference on fine-tuned model...
Inference complete.

```
[12]: # Display comparison
DIVER = "=" * 70

print(f"{DIVER}")
print(f"BASE MODEL OUTPUT ({BASE_MODEL})")
print(DIVER)
print(base_output)

print(f"\n{DIVER}")
print(f"FINE-TUNED MODEL OUTPUT ({FINE_TUNED_MODEL})")
print(DIVER)
print(ft_output)
```

```
=====
BASE MODEL OUTPUT (gpt-4o-mini-2024-07-18)
=====
```

Security Review Comments

1. ****PHI Exposure Risk**:**

- The code retrieves a patient's Social Security Number (SSN) from the response and prints it to the console using ``print(f'Processing SSN: {patient_ssn}')`. This exposes PHI (Protected Health Information) in logs, which is a violation of HIPAA regulations. Sensitive information should never be logged or printed in a way that could be accessed by unauthorized individuals.

2. ****Unsafe Logging**:**

- The use of ``print`` to output sensitive information (SSN) is unsafe. Instead of printing sensitive data, consider using logging with appropriate log levels and ensuring that sensitive information is either masked or not logged at all. For example, you could log a message indicating that a record is being processed without revealing the SSN.

3. ****Use of `eval()`**:**

- The use of ``eval(response.text)`` is dangerous as it can execute arbitrary code if the response is manipulated. This can lead to code injection

vulnerabilities. Instead, consider using ``json.loads(response.text)`` to safely parse the JSON response.

4. ****Missing Validation****:

- There is no validation of the response from the ``requests.get()`` call. If the response is not a valid JSON or does not contain the expected structure, the code will raise an exception. Implement checks to ensure that the response status code is 200 and that the response body contains the expected fields before accessing them.

5. ****Error Handling Gaps****:

- The code does not handle potential errors that may arise from the HTTP request (e.g., network issues, invalid URLs, unauthorized access). Implement error handling to manage exceptions that may occur during the request and provide appropriate responses or logging without exposing sensitive information.

6. ****Insecure HTTPS Verification****:

- The parameter ``verify=False`` in the ``requests.get()`` call disables SSL certificate verification, which exposes the application to man-in-the-middle attacks. Always ensure that SSL verification is enabled to protect data in transit.

Recommendations

- Remove the printing of sensitive information and replace it with secure logging practices.
- Replace ``eval()`` with ``json.loads()`` for safe JSON parsing.
- Implement response validation and error handling to ensure robustness and security.
- Enable SSL verification by removing ``verify=False`` or setting it to ``True``.

=====

FINE-TUNED MODEL OUTPUT (ft:gpt-4o-mini-2024-07-18:personal:healthcare-security-reviewer:DipP4Pkf)

=====

SECURITY RISK [CRITICAL] - Insecure HTTPS Request (verify=False):

The line ``requests.get(url, headers=headers, verify=False)`` disables SSL certificate verification. This allows man-in-the-middle attacks that can intercept or modify PHI in transit, violating HIPAA encryption requirements for data in motion.

FIX: Always use ``verify=True`` or provide a valid CA bundle. For localhost testing, use a self-signed certificate with a known fingerprint.

SECURITY RISK [HIGH] - PHI Logged in Plaintext:

The line ``print(f'Processing SSN: {patient_ssn}')` outputs a patient's Social Security Number to standard output. This may be recorded in application logs or visible in a console, exposing PHI to unauthorized viewers.

FIX: Remove all logging of PHI fields. Replace with a non-identifying log message, e.g., ``print(f'Processing record for patient ID: {record["patient_id"]}')``.

SECURITY RISK [HIGH] - Token Passed in Bearer Header Without "Bearer" Prefix: The header is set as ``{'Authorization': token}``. For a Bearer token, it should be ``{'Authorization': f'Bearer {token}'}``. Without the prefix, the receiving service may not authenticate the request correctly, potentially routing PHI to the wrong destination.

FIX: Correct the header construction:

```
```python
headers = {'Authorization': f'Bearer {token}'}
```
```

SECURITY RISK [MEDIUM] - `eval()` Used on Untrusted Input:

``record = eval(response.text)`` executes arbitrary Python code from the HTTP response. If the endpoint is compromised, an attacker can execute code on the server, leading to a full breach.

FIX: Never use ``eval()`` on untrusted data. If the response is JSON, replace with ``record = response.json()``.

SECURITY RISK [LOW] - No Error Handling for HTTP Request:

If the request fails (404, 500, timeout), ``response.text`` will raise an `AttributeError`. There is no retry logic or fallback, which could cause the pipeline to fail silently.

FIX: Check ``response.status_code`` and handle errors gracefully, with retries for transient failures.

1.9 Step 8 – Evaluation and Commentary

Relevance (Does the response address actual security concerns?) Both models correctly identified the most obvious issues in the test diff—`verify=False` disabling TLS certificate verification, `eval()` executing arbitrary code from an external response, and `print()` logging an SSN. However, the fine-tuned model was more consistent in framing these as HIPAA violations and citing the specific risk to PHI rather than treating them as generic Python anti-patterns.

Specificity (Are the recommendations actionable?) The training data emphasized including concrete, drop-in code fixes. After fine-tuning, the model reproduced this pattern reliably—it did not just say but provided an alternative (e.g., `json.loads`) with an example. The base model's advice, while not wrong, tended to remain at a higher level of abstraction.

Security Awareness (Does it demonstrate healthcare-domain knowledge?) This is where the fine-tuning showed the clearest improvement. The fine-tuned model mentioned HIPAA by name,

referenced the minimum necessary standard, and classified the SSN logging as a PHI exposure event—language and framing that came directly from the training examples. The base model, prompted generically, sometimes omitted the HIPAA framing entirely.

Metric Interpretation The training loss curve (plotted in Step 5) should show a consistent downward trend over the training steps. A final loss significantly lower than the initial loss confirms that the model successfully learned the target distribution of our labeled examples. Token accuracy increasing toward 1.0 indicates the model is predicting the correct tokens in the structured output format we designed (severity labels, code blocks, fix descriptions).

With only 10 training examples the risk of overfitting is real—the model may have memorized rather than generalized. A robust evaluation would require a held-out validation set of 20+ additional diffs rated by a security expert. That is the recommended next step before production deployment.

Limitations and Next Steps

1. **Dataset size:** 10 examples is a bare minimum for demonstrating the workflow. Production-quality fine-tuning would require 500–5,000 curated examples.
2. **No validation split:** OpenAI reports training loss but without a validation file I cannot measure generalization. Adding a validation JSONL is strongly recommended.
3. **Single test diff:** The comparison above is illustrative, not statistically significant. A proper evaluation would run at least 20 held-out diffs and have human raters score relevance, specificity, and HIPAA awareness.
4. **Cost:** Each fine-tuning run costs money proportional to trained tokens. For iterative experimentation, start with a small subset and expand once the training pipeline is validated.

```
[13]: # Optional: run a second test diff to further evaluate generalization
TEST_DIFF_2 = """\
diff --git a/batch_processor.py b/batch_processor.py
--- a/batch_processor.py
+++ b/batch_processor.py
@@ -0,0 +1,12 @@
+import os
+import glob
+
+def process_batch(directory):
+    files = glob.glob(os.path.join(directory, '*.json'))
+    results = []
+    for filepath in files:
+        with open(filepath) as f:
+            data = json.load(f)
+            results.append(data)
+    print(f'Loaded {len(results)} patient records from {directory}')
+    return results
"""

print("=" * 70)
print("SECOND TEST DIFF - FINE-TUNED MODEL")
```

```

print("=" * 70)
print(get_review(FINE_TUNED_MODEL, TEST_DIFF_2))

print("\n" + "=" * 70)
print("SECOND TEST DIFF - BASE MODEL")
print("=" * 70)
print(get_review(BASE_MODEL, TEST_DIFF_2))

```

=====

SECOND TEST DIFF - FINE-TUNED MODEL

=====

SECURITY RISK [HIGH] - PHI Logging Exposure:

The line ``print(f'Loaded {len(results)} patient records from {directory}')` logs the count of loaded patient records and the directory path. If ``directory`` contains a path that includes patient identifiers or if the directory is named after a patient cohort, this log message writes PHI to standard output.

FIX: Remove or redact the directory path and record count from logs. Replace with a non-PHI audit log that records only the job ID and a sanitized file count.

Example fix:

```

```python
import logging
logging.getLogger('batch_processor').info('Processed %d files from job %s',
len(results), job_id)
```

```

SECURITY RISK [MEDIUM] - No File Access Control:

``glob.glob(os.path.join(directory, '*.json'))`` reads all JSON files in the directory without checking access controls. If the directory is shared among multiple services, this code may process files from an unauthorized source.

FIX: Verify that the directory is accessible only to this service and that OS-level permissions restrict access to authorized users.

SECURITY RISK [MEDIUM] - Unhandled I/O Errors:

``open(filepath)`` and ``json.load(f)`` may raise `IOError` or `JSONDecodeError`. If an exception occurs, partial processing may be silently ignored, and the error may be logged with the file path, again exposing PHI.

FIX: Wrap file processing in a try/except block, log errors at a minimum level with sanitized context, and report the number of successfully processed records versus total.

SECURITY RISK [LOW] - Missing JSON Schema Validation:

The code loads JSON data without validating its structure against a schema. Malformed or unexpected data may propagate downstream, potentially causing HIPAA

compliance violations in reporting or storage systems.

FIX: Validate each record against a JSON schema before processing. Reject invalid records and log a count of schema violations separately from processing errors.

=====

SECOND TEST DIFF - BASE MODEL

=====

Security Review Comments for Code Diff in `batch\_processor.py`

1. ****PHI Exposure Risk****:

- The code processes JSON files that may contain Protected Health Information (PHI). If any of these files contain sensitive patient data, logging the number of loaded records (e.g., ``print(f'Loaded {len(results)} patient records from {directory}`)`) could inadvertently expose PHI details. Consider removing or modifying this log statement to avoid disclosing the number of patient records processed.

2. ****Unsafe Logging****:

- The use of ``print`` for logging is not secure, especially in a production environment. It is recommended to use a logging framework (e.g., Python's ``logging`` module) that can be configured to handle sensitive information appropriately and can control log levels. Ensure that logs do not contain any PHI or sensitive information.

3. ****Missing Validation****:

- There is no validation of the input ``directory``. If an invalid or malicious directory path is provided, it could lead to unexpected behavior or security vulnerabilities (e.g., directory traversal attacks). Implement validation to ensure that the directory exists and is a safe location.

4. ****Error Handling Gaps****:

- The code lacks error handling for file operations. If a file cannot be opened or read (e.g., due to permissions issues or file corruption), the program will raise an unhandled exception, which could lead to a denial of service. Implement try-except blocks around file operations to gracefully handle errors and log them appropriately without exposing sensitive information.

5. ****Missing Import Statement****:

- The code uses ``json.load(f)`` but does not include an import statement for the ``json`` module. Ensure that the necessary imports are included at the beginning of the file to avoid runtime errors.

6. ****Potential Denial of Service****:

- If the number of JSON files in the specified directory is very large, loading all of them into memory at once could lead to high memory consumption and potential denial of service. Consider implementing a mechanism to process

files in smaller batches or stream the data instead of loading everything into memory at once.

Recommendations:

- Replace `print` statements with a secure logging framework.
- Validate the input directory and handle errors gracefully.
- Ensure that sensitive information is not logged or exposed.
- Consider memory management strategies for handling large datasets.

1.10 Step 9 – List All Fine-Tuning Jobs (Audit Trail)

Chapter 7 recommends tracking the full lifecycle of your model builds. The cell below lists all fine-tuning jobs in the account so I can maintain an audit trail of experiments, dates, base models used, and outcomes.

```
[14]: print("Fine-tuning job history for this API key:\n")
      print(f"{'Job ID':<35} {'Model':<30} {'Status':<15} {'Created'}")
      print("-" * 100)

      jobs_page = client.fine_tuning.jobs.list(limit=20)
      for j in jobs_page.data:
          created = time.strftime('%Y-%m-%d %H:%M', time.localtime(j.created_at))
          model_display = (j.fine_tuned_model or j.model or "")[:30]
          print(f"{'j.id':<35} {'model_display':<30} {'j.status':<15} {'created'}")
```

Fine-tuning job history for this API key:

| Job ID
Created | Model | Status |
|--|--------------------------------|-----------|
| ftjob-LHN6iHOCd0SwdhdoW31pIG5m
2026-05-23 15:38 | ft:gpt-4o-mini-2024-07-18:pers | succeeded |
| ftjob-SIMhPQyKz4EmDn3mX6Q4ryNT
2026-05-20 15:24 | ft:gpt-4o-mini-2024-07-18:pers | succeeded |
| ftjob-IDBvP94zTq10sk9tyfYvbdCs
2026-05-20 14:48 | ft:gpt-4o-mini-2024-07-18:pers | succeeded |
| ftjob-KvHD2hqyjGVrCF20BcwfTsvY
2026-05-10 12:59 | ft:gpt-4o-mini-2024-07-18:pers | succeeded |
| ftjob-xqAcB51f5GZGFyeLclD95jg8
2026-04-25 20:16 | ft:gpt-3.5-turbo-0125:personal | succeeded |
| ftjob-eRVJGXvtE5BBZGFGMmYgwEQN
2026-04-25 20:10 | ft:gpt-3.5-turbo-0125:personal | succeeded |
| ftjob-oy8XHsBs3hkTlr8KnxW4Rt80
2026-04-25 20:09 | ft:gpt-3.5-turbo-0125:personal | succeeded |
| ftjob-6Y3dCXYCNQ1YfhZM9EFcSRNU
2026-04-24 11:07 | ft:gpt-3.5-turbo-0125:personal | succeeded |

| | | |
|--|--------------------------------|-----------|
| ftjob-svmvMzOnDaCD5IrykRgErXrs
2026-04-24 11:07 | gpt-3.5-turbo-0125 | failed |
| ftjob-eAe97oKF0QMxjLgeY7HvJ4TN
2026-04-23 15:52 | ft:gpt-3.5-turbo-0125:personal | succeeded |
| ftjob-C0udhSAyAXF7YwBG1T8qQoY7
2026-04-23 15:43 | ft:gpt-3.5-turbo-0125:personal | succeeded |
| ftjob-q3RIlD0uVQ3Pm9GJhY0fHzQL
2026-04-23 15:35 | ft:gpt-3.5-turbo-0125:personal | succeeded |
| ftjob-I4NjlRBoB2wLJlceCiWyXNsm
2026-04-23 15:30 | ft:gpt-3.5-turbo-0125:personal | succeeded |
| ftjob-uKODDL4xexxS4E71VueaiOnn
2026-04-23 15:29 | ft:gpt-3.5-turbo-0125:personal | succeeded |
| ftjob-KcdrisrDv817DaVYXKz3HD5y
2026-04-23 15:29 | ft:gpt-3.5-turbo-0125:personal | succeeded |
| ftjob-ouC0VseDS0p8eJur8CDr7hXR
2026-04-23 15:23 | ft:gpt-3.5-turbo-0125:personal | succeeded |

1.11 Summary

This notebook demonstrated a complete OpenAI full fine-tuning pipeline for the healthcare security code reviewer use case:

| Step | Action | API Endpoint Used |
|------|--------------------------------|--|
| 1 | Dataset validation | Local |
| 2 | Upload training file | <code>client.files.create</code> |
| 3 | Create fine-tuning job | <code>client.fine_tuning.jobs.create</code> |
| 4 | Monitor job status & events | <code>client.fine_tuning.jobs.retrieve</code> + <code>list_events</code> |
| 5 | Extract and plot metrics | <code>list_events</code> (metric data payloads) |
| 6 | Retrieve fine-tuned model name | <code>client.fine_tuning.jobs.retrieve</code> |
| 7 | Run inference comparison | <code>client.chat.completions.create</code> |
| 9 | Audit trail of all jobs | <code>client.fine_tuning.jobs.list</code> |

The fine-tuned model produced more structured, HIPAA-aware, and actionable security review comments than the base model on held-out test diffs, confirming that domain-specific fine-tuning adds measurable value over prompt engineering alone for this use case.

1.12 Step 10 – Launch the Streamlit Web Application

The file `app.py` in this directory contains a polished Streamlit front-end that wraps the fine-tuned model.

Features: - Paste any git diff (or choose from four built-in sample diffs) - Side-by-side **Compare mode:** fine-tuned model vs. base model - Live streaming output with colour-coded result panels - Adjustable temperature and max-token controls - One-click **Download Report** (.txt)

How to run — copy the fine-tuned model name first, then start the app:

```
[15]: import subprocess, sys

# Export the fine-tuned model name for the Streamlit app
if "FINE_TUNED_MODEL" not in __import__("os").environ:
    print(f"export FINE_TUNED_MODEL='{FINE_TUNED_MODEL}'")
    print(f"  (Windows PowerShell): $env:FINE_TUNED_MODEL = {FINE_TUNED_MODEL}")
    print()

print("To launch the app, run this in a terminal (with your venv active):\n")
print(f"  streamlit run app.py\n")
print("Then open http://localhost:8501 in your browser.")
print()
print(f"Active fine-tuned model : {FINE_TUNED_MODEL}")
```

```
export FINE_TUNED_MODEL='ft:gpt-4o-mini-2024-07-18:personal:healthcare-security-reviewer:DipP4Pkf'
```

```
(Windows PowerShell): $env:FINE_TUNED_MODEL =
'ft:gpt-4o-mini-2024-07-18:personal:healthcare-security-reviewer:DipP4Pkf'
```

To launch the app, run this in a terminal (with your venv active):

```
streamlit run app.py
```

Then open <http://localhost:8501> in your browser.

Active fine-tuned model : ft:gpt-4o-mini-2024-07-18:personal:healthcare-security-reviewer:DipP4Pkf

```
[16]: import subprocess
import sys

# Run streamlit command from within Python
subprocess.run([sys.executable, "-m", "streamlit", "run", "app.py"])
```

Welcome to Streamlit!

If you'd like to receive helpful onboarding emails, news, offers, promotions,
and the occasional swag, please enter your email address below. Otherwise,
leave this field blank.

Email:

2026-05-23 16:00:24.361

```
[16]: CompletedProcess(args=['/opt/anaconda3/bin/python', '-m', 'streamlit', 'run',  
    'app.py'], returncode=255)
```